C4375 Assignment

https://github.com/Nianniande/CS-4375-Assignment1

Andy Weng
AXW220034

TODO: Fill in your name and details above. If not filled correctly, we will subtract 2pt.
TODO: It is highly suggested to use the template for reports. You are free to use other softwares (e.g. Microsoft Word, Google Doc), but we prefer you follow the structure below.

1 Introduction and Data (5pt)

In this project, we study the performance of two neural network architectures. Using Feedforward Neural Network (FFNN) and Recurrent Neural Network (RNN) and analysis a Yelp reviews. The task is to predict the rating of a review that ranges between 1 to 5.

The main experiment include training the FFNN and RNN on the training set with different hyperparameter settings. Evaluating the performance on a validation set to measure generalization. Compare models to understand the different between FFNN and RNN. Analysis the impact of increasing hidden size and other changes.

The datasets provided including training.json, validation.json, and test.json. The training set is used to learn model, the validation set is used to tune the hyperparameters and monitor the performance during the training, and the test set is for final evaluation.

Training Set	8000 examples
Validation Set	800 examples
Test Set	800 examples

Model	Hidden Dim	Epochs	Validation Accuracy
FFNN	32	10	0.63125
RNN	32	10	0.5775

I train both FFNN and RNN using a hidden dimension of 32. The FFNN model achieved it best validation accuracy of 0.63125 at epoch 5. The RNN model achieved its best validation accuracy of 0.5775 at epoch 2 and stop at epoch 3 to prevent overfitting.

Overall, the FFNN model performed better than the RNN model on this dataset.

2 Implementations (45pt)

2.1 FFNN (20pt)

```
def forward(self, input_vector):
    # [to fill] obtain first hidden layer representation
    h = self.activation(self.W1(input_vector))
    # [to fill] obtain output layer representation
    z = self.W2(h)
    # [to fill] obtain probability dist.
    predicted_vector = self.softmax(z)
    return predicted_vector
```

For the first to fill in the forward function, I applied a linear transformation to the input vector using the layer W1 and then pass the result through a ReLU activation function. By producing

the hidden layer representation, it captures the learned feature from the input and introduce nonlinear so the network can model complex patterns.

For the second to fill in the forward function, I compute the output layer representation z by applying a second linear transformation W2 to the hidden layer h to produce raw scores for each of the five rating classes.

For the last to fill, I applied a LogSoftmax function to z to convert the logits to log-probabilities and takes the output for comparing against the true label.

The implementation uses PyTorch for defining the network, layers, activations, loss and optimization, Numpy is used for general array operations, JSON for reading the data, argparse for parsing command line argument, random for shuffling, time for measuring training and validation time, and tqdm for progress bars. No additional tools or libraries were required.

Other important part of code includes using stochastic gradient descent (SGD) with momentum as the optimizer.

```
optimizer = optim.SGD(model.parameters(),lr=0.01, momentum=0.9)
```

This updates the model weights based on the gradient while using past updates to speed up learning and avoid of getting stuck.

```
print("========== Vectorizing data ==========")
train_data = convert_to_vector_representation(train_data, word2index)
valid_data = convert_to_vector_representation(valid_data, word2index)
```

This input vectorization turns each text review into a numeric bag-of-words vector that the network can use.

```
# fix random seeds
random.seed(42)
torch.manual_seed(42)
```

The random seed are set to make the result reproducible.

```
for epoch in range(args.epochs):
    model.train()
    optimizer.zero_grad()
    loss = None
    correct = 0
    total = 0
    start_time = time.time()
    print("Training started for epoch {}".format(epoch + 1))
    random.shuffle(train_data) # Good practice to shuffle order of tra
    minibatch_size = 16
    N = len(train_data)
    for minibatch_index in tqdm(range(N // minibatch_size)):
```

The training loop goes through a fixed number of epochs and shuffles the training data for each epoch to help generalization and uses minibatches to update weight efficiently.

```
for minibatch_index in tqdm(range(N // minibatch_size)):
    optimizer.zero_grad()
    loss = None
    for example_index in range(minibatch_size):
        input_vector, gold_label = valid_data[minibatch_index * minibatch_size +
            example_index]
        predicted_vector = model(input_vector)
        predicted_label = torch.argmax(predicted_vector)
        correct += int(predicted_label == gold_label)
        total += 1
        example_loss = model.compute_Loss(predicted_vector.view(1,-1),
            torch.tensor([gold_label]))
        if loss is None:
            loss = example_loss
        else:
            loss += example_loss
    loss = loss / minibatch_size
print("Validation completed for epoch {}".format(epoch + 1))
print("Validation accuracy for epoch {}: {}".format(epoch + 1, correct / total))
print("Validation time for this epoch: {}".format(time.time() - start_time))
```

Stopping and evaluation are handled by checking the model on the validation set after each epoch to see how well it is learning. Training just stop after a fix number of epchos.

```
self.softmax = nn.LogSoftmax() # The softmax function that converts vectors into
    probability distributions; computes log probabilities for computational
    benefits
self.loss = nn.NLLLoss() # The cross-entropy/negative log likelihood loss taught
    in class
```

Finally, the loss function works with LogSoftmax to measure the difference between the predicted rating and the true rating and guide the optimizer to improve the model.

2.2 RNN (25pt)

```
def forward(self, inputs):
    # [to fill] obtain hidden layer representation
    outputs, hidden = self.rnn(inputs)
    # [to fill] obtain output layer representations
    z = self.W(outputs)
    # [to fill] sum over output
    z_sum = torch.sum(z, dim=0)
    # [to fill] obtain probability dist.
    predicted_vector = self.softmax(z_sum)
    return predicted_vector
```

For the first to fill in the forward function, I implemented the hidden states for each word in the input sequence using a single-layer RNN with a tanh activation.

For the second to fill, a linear layer W maps each hidden state to a raw score (logits) for the five rating.

For the third to fill, since the input is a sequence, the logits are summed over the sequence dimension to produce a single output vector.

Finally for the fourth to fill, LogSoftmax is applied to convert logits to log probabilities and take the output for compare with the true label.

The tools and libraries are similar to FFNN where it uses PyTorch for defining the network, layers, activations, loss and optimization, Numpy is used for general array operations, JSON for

reading the data, argparse for parsing command line argument, random for shuffling, time for measuring training and validation time, and tqdm for progress bars. No additional tools or libraries were required.

```
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

For training, RNN uses the Adam optimize instead of SGD that were used for FFNN.

```
# Look up word embedding dictionary
vectors = [word_embedding[i.lower()] if i.lower() in word_embedding.keys() else
    word_embedding['unk'] for i in input_words ]

# Transform the input into required shape
vectors = torch.tensor(vectors).view(len(vectors), 1, -1)
output = model(vectors)
```

Unlike FFNN which takes a single bag-of-words vector for each document, the RNN processes sequences of word embeddings. Each word in the input is converted to a vector using pretrained embeddings and unknow words are mapped to <UNK>.

```
if validation_accuracy < last_validation_accuracy and trainning_accuracy >
    last_train_accuracy:
    stopping_condition=True
    print("Training done to avoid overfitting!")
    print("Best validation accuracy is:", last_validation_accuracy)
else:
    last_validation_accuracy = validation_accuracy
    last_train_accuracy = trainning_accuracy
```

For the stopping conditions, RNN has a condition to prevent overfitting. The loop will monitor validation accuracy and stop if the accuracy decreases while training accuracy continues to improve. In FFNN, training just stops after a fixed number of epochs.

For training, RNN capture word order and contextual information where FFNN only models word occurrence.

Finally for loss function, it the same as FFNN, which use NLLLoss with LogSoftmax output to compute the difference between the predicted rating and the true rating.

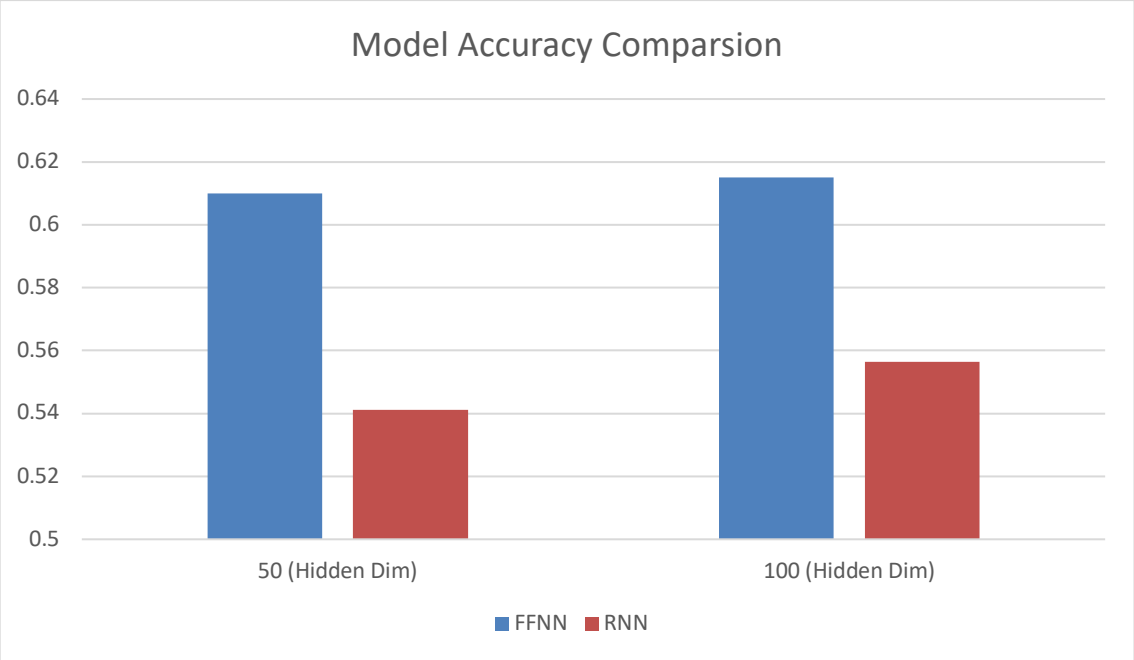
3 Experiments and Results (45pt)

Evaluations (15pt)

To evaluate the models, we used accuracy as the main metric to detect the performance of the models. This measures the percentage of correctly predicted review ratings. For each example, the model output a vector of log probabilities over the five classes, and the predicted class is chosen as the one with the highest log probability using torch.argmax. The evaluation is performed on both eh training set and the validation set to monitor learning and generalization.

Results (30pt)

Model	Hidden Dim	Epoch	Validation Accuracy
FFNN	50	6	0.610
FFNN	100	9	0.615
RNN	50	6	0.54125
RNN	100	1	0.55635



For both Model, increasing hidden dimensional units generally improves the validation accuracy. Base on the table FFNN shows a smooth improvement across epochs, while RNN tends to get overfit faster.

For the report, we pick the epoch with the highest validation accuracy for each model. Based on the table, FFNN outperform RNN for both 50 and 100 hidden dimensions in term of validation accuracy. One concern of RNN's validation accuracy being lower is because it likely need to epochs to learn and the overfitting handling often make the training stop early.

4 Analysis (bonus: 1pt)

TODO:

- (1pt) Plot the learning curve of your best system. The curve should include the training loss and development set accuracy by epoch.
- (1pt) Error analysis. List some (one or more than one) error examples and provide some analysis. How might you improve the system?

5 Conclusion and Others (5pt)

I was responsible for implement the forward function of both FFNN and RNN models and all the experiments, result table, and comparison analysis.

The assignment took approximately 10 hours to complete, including the setting up of the environment, filling the forward function, understanding the function of code, running the training, and analyzing the results of the training. The difficulty of the project was medium, it needs a lot of researching to understand the code and it function. The most challenge part of the project was to understand the code and knows that each part do and how the models was being trained. Another part that took me sometimes are the setting up the environment in Conda for training. One improvement of this project is to give a specific instruction of setting up the environment of for training.